World-Model Based Reinforcement Learning for Hockey: DreamerV3 with Adaptive Opponent Curriculum

Carl Kueschall
University of Tübingen

Winter Semester 2025/26

1 Introduction

Note: Originally a two-person project; co-author exmatriculated. Work completed by remaining author.

The hockey environment [5] presents a challenging continuous control problem in an adversarial setting. Two agents compete in an air-hockey game with 18-dimensional state observations encoding positions, velocities, and puck state, while each player controls 4 continuous action dimensions. Episodes last up to 250 timesteps with extremely sparse rewards: $+10$ for scoring, -10 for conceding, and 0 otherwise. This sparsity makes credit assignment difficult—agents must learn complex behaviors (positioning, puck control, shooting, defending) from reward signals that occur only at episode boundaries.

I implement DreamerV3 [3], a world-model based reinforcement learning algorithm that addresses sparse rewards by learning entirely in "imagination." The agent first learns a world model from real experience, then trains its policy through simulated rollouts in learned latent space. My implementation builds upon the NaturalDreamer codebase[1], with significant modifications for hockey: Two-Hot Symlog for sparse reward prediction, DreamSmooth [6] temporal reward smoothing, inverse-frequency sparse-event loss weighting in world-model reward learning, PFSP self-play, and an adaptive opponent curriculum. A key project realization was that late-stage performance depends less on fixed bots and more on recursive opponent-pool management and curated league diversity.

Contributions: (1) DreamerV3 hockey implementation with Two-Hot Symlog and DreamSmooth; (2) adaptive opponent curriculum with effective compute split of roughly 20% anchor-only, 20% bootstrap self-play, and 60% league-dominant self-play; (3) robust checkpoint-selection tooling using worst-case and quantile metrics, not only mean win rate; (4) strong late checkpoints (556k/612k/634k) that consistently outperform older baselines in summit cross-play.

2 Methods

2.1 DreamerV3: World-Model Based RL

2.1.1 Background

DreamerV3 [3] learns a world model that predicts future states and rewards from actions, then trains actor-critic policies entirely through imagined rollouts in learned latent space. This is particularly suited for sparse-reward environments: the world model propagates reward information backward through

[1] `https://github.com/InexperiencedMe/NaturalDreamer`

imagined trajectories, enabling credit assignment even when real rewards are rare. Our implementation adapts this for the hockey domain, addressing (1) extremely sparse goal rewards, (2) continuous 4D action space, and (3) adversarial dynamics.

2.1.2 World Model Architecture

The world model uses a Recurrent State Space Model (RSSM) combining deterministic and stochastic dynamics. The state $s_t = (h_t, z_t)$ consists of a deterministic GRU hidden state $h_t \in \mathbb{R}^{256}$ and stochastic latent z_t (16 categorical variables, 16 classes each). Dynamics: $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$; prior $\hat{z}_t \sim p_\theta(z_t|h_t)$; posterior $z_t \sim q_\phi(z_t|h_t, \text{Enc}(o_t))$. KL divergence with free nats thresholding trains the model. I additionally use a continuation head that predicts non-terminal probability and provides the imagination-time continuation factor in return computation.

Two-Hot Symlog. MSE fails on hockey's sparse reward distribution $\{-10, 0, +10\}$. We use Two-Hot Symlog encoding [3]: $\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1)$, discretizing into 255 bins. The two-hot encoding places mass on bins surrounding the target, enabling gradient flow. Applied to both reward and value prediction.

2.1.3 Key Modifications for Hockey

Table 1 summarizes our domain-specific modifications. **DreamSmooth** [6] propagates goal rewards backward: $\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$ with $\alpha = 0.5$, addressing temporal credit assignment for goals at episode boundaries. In addition, I use inverse-frequency weighting for sparse reward events ($|r| > 1$) in reward-head training, capped for stability, to prevent rare goal transitions from being gradient-diluted by the dominant near-zero reward mass.

Modification	Problem Addressed	Implementation
Two-Hot Symlog	MSE fails on sparse rewards	255 bins in symlog space
DreamSmooth	Temporal credit assignment	$\tilde{r}_t = \alpha r_t + (1-\alpha)\tilde{r}_{t+1}$, $\alpha = 0.5$
Sparse-event loss weighting	Rare goal events under-weighted in reward loss	Inverse-frequency weighting for $\|r\| > 1$, clipped for stability
Anchor Opponents	Prevent catastrophic forgetting	10% weak + 10% strong in late phase
Self-Play + PFSP	Limited diversity	48-checkpoint pool, variance-weighted PFSP, periodic insertion

Table 1: Key modifications from baseline DreamerV3 for the hockey domain.

2.1.4 Recursive Opponent-Pool Management

The final training policy is controlled by opponent sampling rather than reward shaping. Let ρ be the anchor probability; then

$$\mathbb{P}(\text{anchor}) = \rho, \quad \mathbb{P}(\text{self-play}) = 1 - \rho.$$

In late runs we set $\rho = 0.2$, yielding an empirical distribution close to 80/10/10 (self-play/weak/strong). This keeps fixed-bot competence while maximizing exposure to diverse, hard opponents. In league runs, PFSP sampling is further blended with a static checkpoint prior from curated league weights, using exponent α (`self_play_prior_alpha`) so higher-quality checkpoints are sampled earlier without disabling PFSP adaptation.

To avoid selecting brittle checkpoints, I rank candidates with robustness-aware metrics from cross-play:

$$\text{RobustScore} = 0.60 \cdot \text{CP}_{\min} + 0.25 \cdot \text{Fixed}_{\text{combined}} + 0.15 \cdot \text{CP}_{\text{mean}},$$

and in later tooling also use CP_{q20} (20th percentile) for stability under non-transitive matchups.

2.1.5 Behavior Learning

Actor: Tanh-squashed Gaussian with $\sigma_{\min} = 0.1$ plus mean-regularization on pre-squash means (`mean_reg_scale`) to reduce tanh-saturation drift. **Critic:** Two-Hot Symlog with slow-critic EMA regularization ($\tau = 0.98$). **Lambda returns:** TD(λ), $\lambda = 0.95$, $\gamma = 0.997$. **Value normalization:** Percentile-based scaling (5th/95th) for advantages. **Actor loss:**

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{s\sim\text{imagine}}\left[\frac{V^{\lambda}(s) - V(s)}{S} + \eta \cdot H[\pi(\cdot|s)]\right] \tag{1}$$

with $\eta = 3 \times 10^{-4}$ fixed.

3 Experimental Evaluation

3.1 From 2-Phase Baseline to Adaptive 3-Stage Curriculum

The early project used a 2-phase recipe (mixed fixed bots, then self-play) and reached a strong fixed-bot baseline at 336k. The final push revealed a clearer compute-allocation structure (Table 2): fixed-bot training is necessary but quickly saturates, while most late gains come from league-dominant self-play with curated opponents.

Stage	Compute Share	Main Opponents	Purpose / Outcome
A: Anchor competence	∼20%	Weak + strong fixed bots	Learn fundamentals and stabilize the world model; fast gains vs. fixed bots.
B: Bootstrap self-play	∼20%	Internal self-play + anchors	Introduce adversarial adaptation; produce stronger checkpoints (e.g., 474k/556k).
C: Curated league self-play	∼60%	80% league, 10% weak, 10% strong	Improve robustness against diverse opponents; yields best late checkpoints (612k/634k).

Table 2: Practical training-time allocation learned during the final push.

A compact visual of this end-to-end training strategy is included in Appendix Figure 4.

In the representative late run (_weak_seed42_20260224_040000, 474k→639k in 10.7h), opponent sampling matched target behavior: self-play ratio ≈ 0.80, anchor ratio ≈ 0.20, and balanced weak/strong anchors. The self-play pool grew to 48 tracked opponents. Pool metrics improved substantially: overall pool win rate reached 0.804 and newest-third win rate rose to 0.725, indicating continued progress against recent opponents.

3.2 Summit Cross-Play and Checkpoint Selection

I evaluate checkpoints with matrix cross-play against fixed bots plus a curated opponent bank. Results show clear improvement over older baselines, but also non-transitive behavior: ranking between 556k/612k/634k changes with seed and opponent set, while all three outperform 357k and 474k.

The main lesson is methodological: fixed-bot win rate saturates early and becomes weakly informative, while `CP_worst` and lower-quantile cross-play metrics identify robustness gaps and guide better pool curation. Across late snapshots, the top checkpoint is not perfectly stable (556k, then 612k, then 634k in different slices), which is consistent with non-transitive matchups and limited per-match episode budgets. I therefore introduced automated shortlist selection (`eval_select_pool.py`) with a two-stage procedure: Stage 1 fast screening (fixed weak/strong + sparse checkpoint cross-play) and Stage 2 dense top-K refinement (optionally bidirectional), followed by greedy quality+diversity selection from pairwise profiles. The resulting 21-checkpoint ranked pool (Appendix Table 3) makes the selection rule explicit. Appendix Figure 6 visualizes the same ranking as a fixed-bot-vs-cross-play tradeoff. Figure 1 summarizes the benchmark chain as a single continuous timeline by merging four runs (from-scratch, self-play-only, league round 1, league round 2) and clipping overlap regions in gradient-step space.

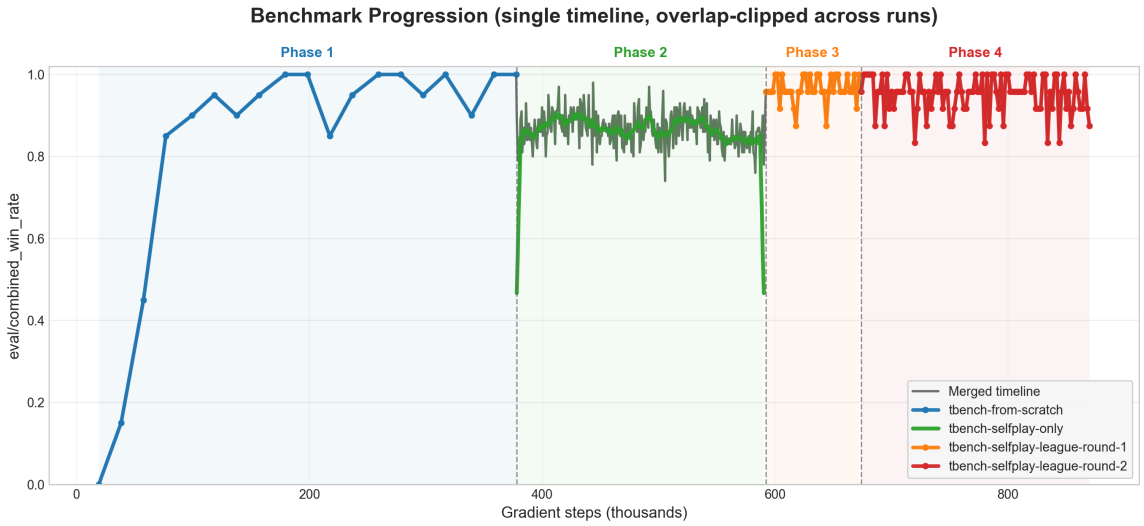


Figure 1: Single-timeline benchmark progression for `eval/combined_win_rate`, constructed from four training runs and overlap-clipped on the x-axis (gradient steps). Colored phase segments indicate the source run of each retained datapoint.

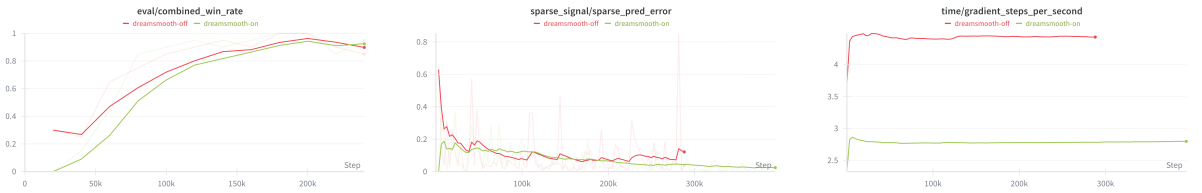
3.3 Self-Play Metrics

Self-play progression is analyzed primarily through quantitative pool metrics from run traces (overall pool win rate and oldest/middle/newest-third breakdowns), while ablation-specific evidence is discussed in the next subsection.

3.4 Ablation Diagnostics Beyond Aggregate Win Rate

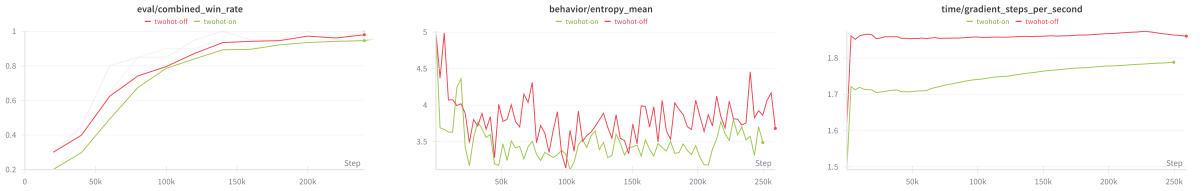
Figures 2 and 3 are interpreted jointly, not by headline win rate alone. In this hockey setup, neither DreamSmooth nor Two-Hot produced a massive jump in `eval/combined_win_rate`; DreamerV3 already performs strongly on the sparse-reward task.

The diagnostic view is still informative. For DreamSmooth, Figure 2(b) shows a cleaner/lower sparse-prediction-error profile, while Figure 2(c) shows a runtime tradeoff (lower gradient-steps/sec). For Two-Hot, Figure 3(a) remains close to Gaussian in final win rate, but Figure 3(b) shows a measurable entropy-shape difference, i.e., a policy-behavior calibration effect rather than a large score jump. All these runs also use sparse-event loss weighting in the reward head (Section 2.1), so rare $|r| > 1$



(a) DreamSmooth: combined win rate (b) DreamSmooth: sparse prediction error (c) DreamSmooth: gradient steps per second

Figure 2: DreamSmooth ablation diagnostics beyond headline win rate.



(a) Two-Hot: combined win rate (b) Two-Hot: entropy mean (c) Two-Hot: gradient steps per second

Figure 3: Two-Hot vs. Gaussian-head diagnostics (performance, entropy behavior, and throughput).

transitions receive more gradient mass; this class-imbalance handling is part of the baseline and was not isolated as a standalone ablation.

3.5 Fixed-Bot Benchmark Saturation

Fixed-bot performance stayed high throughout: the early 336k baseline reached 93.5% combined (100 episodes/opponent), late 556k/634k checkpoints were both around 96.5–96.7% (30 episodes/opponent matrix snapshots), and the final 692k checkpoint reached 99.0% combined (100 episodes/opponent; 100.0% weak, 98.0% strong). This supports the same conclusion as Figure 1: fixed-bot metrics saturate, while cross-play robustness is the more useful late-stage selector. In the official final tournament, this DreamerV3 agent placed **16th among all agents** and **11th among teams**.

4 Discussion

Key Findings and Lessons Learned. First, ablations indicate that DreamSmooth and Two-Hot did not massively change headline win rate in this single-seed setup, but they still served their intended roles: DreamSmooth improved sparse-signal prediction behavior, and Two-Hot changed policy-entropy behavior in a controlled way. Sparse-event loss weighting further supported rare-goal learning by countering class imbalance in reward-head training. Second, the dominant late-stage factor is opponent curriculum design: after fixed-bot competence is reached, most additional compute should be invested in self-play against a curated league. In my final runs, this effectively became a 20/20/60 allocation (anchor/bootstrap/curated-league) and produced stronger checkpoints than the earlier 2-phase baseline. Third, recursive opponent management (periodic checkpoint insertion, PFSP sampling, and matrix-based curation) is what unlocked peak performance, not further optimization on weak/strong bots.

Post-hoc algorithm-fit diagnosis. To understand why this DreamerV3 implementation was competitive but not top-ranked in the tournament, I performed a targeted literature review after the final runs. The core message is that DreamerV3 is strongest when sample efficiency and high-dimensional modeling dominate, while in low-dimensional, fast-simulator continuous control, asymptotic performance is often

led by strong model-free methods such as TD3/SAC [3, 1, 2]. This is consistent with my hockey setting and with newer world-model evidence showing that architecture and planning style matter strongly in continuous control (e.g., TD-MPC2 vs. DreamerV3) [4]. I include this reflection to make clear that the outcome was analyzed scientifically, not only empirically.

Limitations. Matrix evaluations still use modest episode budgets in many snapshots (often 30), so ranking among close late checkpoints has variance. Non-transitivity remains visible, and some tooling constraints (e.g., incompatible legacy checkpoints) required explicit filtering. The sparse-event loss weighting feature was also not isolated with a dedicated multi-seed ablation in the final phase.

Future Work. Run higher-confidence bidirectional matrix evaluations (100+ episodes per matchup, multiple seeds), maintain a continuously refreshed multi-seed league, and integrate online tournament feedback into checkpoint promotion criteria.

5 AI Usage Declaration

My AI-use philosophy had three layers. First, I used agentic systems (primarily Claude Code) for discussion, debate, decision support, concept learning, and targeted paper discovery. Second, I used AI autocomplete for implementing core logic (agent internals, training loop, architecture-level essentials), followed by manual review and verification. Third, I used Claude Code for low-learning-value "dirty work" (repetitive scripting, formatting, and pipeline glue). The decision matrix is shown in Appendix Figure 5.

References

[1] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pages 1587–1596. PMLR, 2018.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.

[3] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2024.

[4] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*, 2024.

[5] G. Martius, P. Kolev, M. Zhobro, and S. Ramen. Hockey environment. `https://github.com/martius-lab/hockey-env`, 2024.

[6] V. Wu, J. Aslanides, J. Ba, and D. Hafner. Dreamsmooth: Improving model-based reinforcement learning via reward smoothing. *arXiv preprint arXiv:2311.01450*, 2023.

A Appendix

A.1 Training Flow Visuals

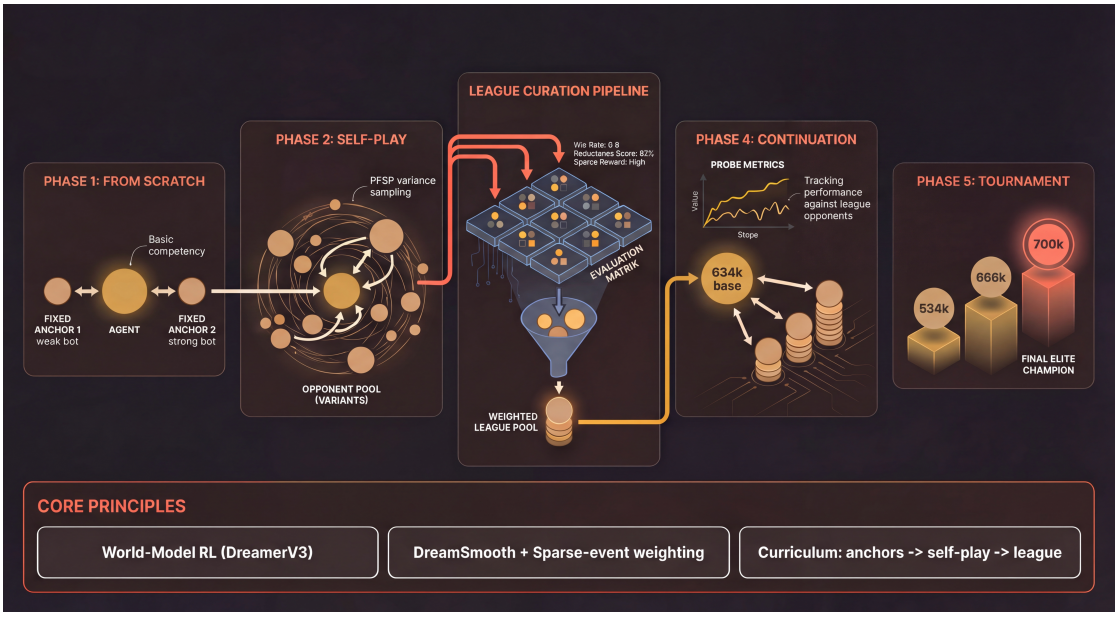


Figure 4: Holistic training-phase visual from scratch to final tournament checkpoint selection.

A.2 AI-Use Decision Matrix

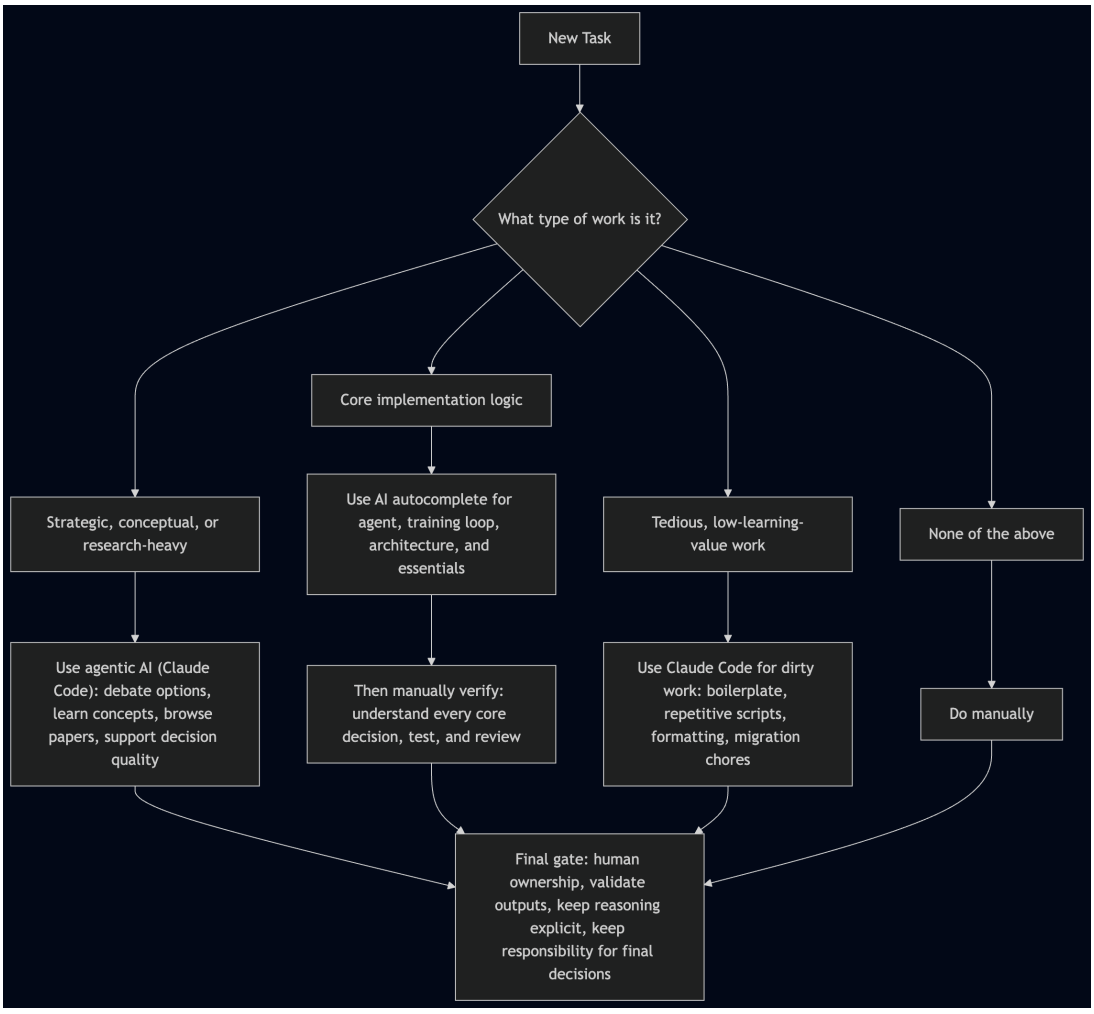


Figure 5: AI-use decision matrix (rendered chart).

A.3 Full League-Pool Ranking

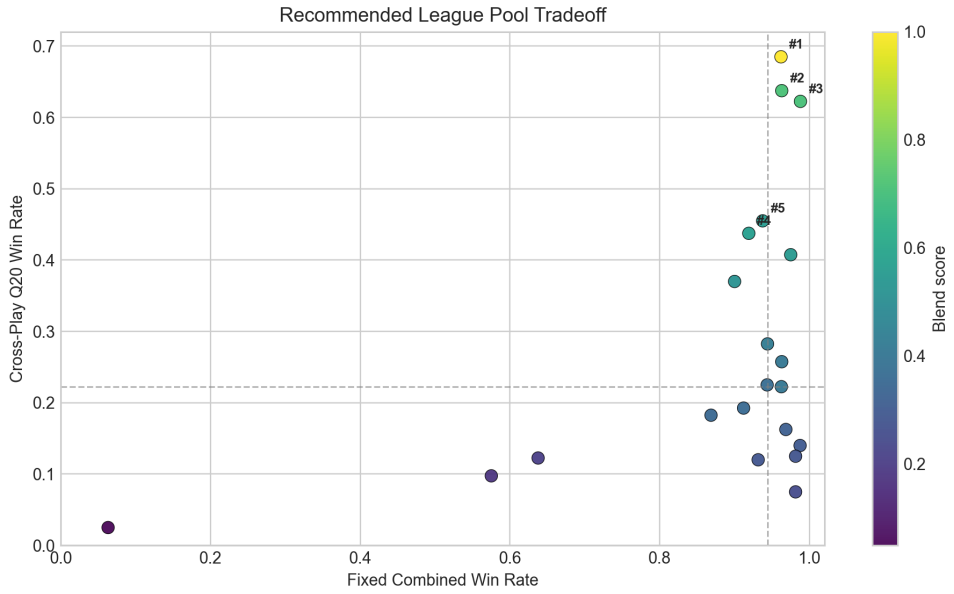


Figure 6: Recommended-pool tradeoff view. The x-axis shows fixed-bot strength and the y-axis shows cross-play robustness (`CP_Q20`); color indicates blend score used for sampling priority.

Table 3: Full 21-checkpoint ranking used to build the curated league pool.

Rank	Checkpoint	Blend	Quality	Diversity	CP Q20	CP Worst	CP Mean	Fixed Combined
1	634k.pth	1.0000	0.7481	1.0000	0.6850	0.5250	0.7712	0.9620
2	612k.pth	0.7111	0.7101	0.0054	0.6375	0.2750	0.7325	0.9630
3	622k.pth	0.7093	0.7087	0.0039	0.6225	0.4625	0.7481	0.9880
4	results_checkpoints__weak_seed42_20260224_074312_546k.pth	0.5678	0.5708	0.0207	0.4375	0.2375	0.6819	0.9190
5	556k.pth	0.5552	0.5572	0.0277	0.4550	0.0125	0.5744	0.9375
6	results_checkpoints__weak_seed42_20260224_013546_558k.pth	0.5496	0.5557	0.0118	0.4075	0.3250	0.6600	0.9750
7	results_checkpoints_summit-local-league80-20260224_010504_weak_seed42_20260224_010505_534k.pth	0.5165	0.5245	0.0115	0.3700	0.2000	0.6700	0.9000
8	results_checkpoints__weak_seed42_20260223_031428_456k.pth	0.4258	0.4306	0.0454	0.2825	0.2000	0.4781	0.9440
9	results_checkpoints_DreamerV3-mixed-selfplay-seed42_weak_seed42_20260124_144802_220k.pth	0.4090	0.4177	0.0330	0.2225	0.0625	0.5594	0.9625
10	results_checkpoints__weak_seed42_20260223_040357_534k.pth	0.4070	0.4202	0.0143	0.2575	0.0375	0.4850	0.9630
11	results_checkpoints__weak_seed42_20260223_031358_406k.pth	0.3673	0.3815	0.0190	0.2250	0.1000	0.4200	0.9435
12	results_checkpoints__weak_seed42_20260224_074312_480k.pth	0.3548	0.3700	0.0181	0.1925	0.0500	0.4706	0.9120
13	results_checkpoints__weak_seed42_20260224_040000_476k.pth	0.3467	0.3629	0.0154	0.1825	0.0375	0.4925	0.8685
14	results_checkpoints_dreamer_weak_seed43_20260125_133552_240k.pth	0.3144	0.3284	0.0320	0.1625	0.0000	0.3425	0.9685
15	results_checkpoints_balanced-finetune-DreamerV3-final-finetune-seed43_weak_seed43_20260125_230625_266k.pth	0.2995	0.3184	0.0147	0.1400	0.0125	0.3450	0.9875
16	357k_0130_193448.pth	0.2796	0.2974	0.0243	0.1200	0.0375	0.3425	0.9315
17	results_checkpoints_dreamer_weak_seed43_20260125_133536_250k.pth	0.2788	0.2996	0.0114	0.1250	0.0125	0.3094	0.9815
18	results_checkpoints_balanced-finetune-DreamerV3-final-finetune-seed43_weak_seed43_20260125_215906_296k.pth	0.2445	0.2653	0.0189	0.0750	0.0000	0.2925	0.9815
19	results_checkpoints_DreamerV3-dreamsmooth-seed42_weak_seed42_20260122_000848_120k.pth	0.2058	0.2187	0.0615	0.1225	0.0375	0.1981	0.6375
20	results_checkpoints_DreamerV3-small-weak-seed43_weak_seed43_20260120_122246_50k.pth	0.1772	0.1952	0.0462	0.0975	0.0500	0.2019	0.5750
21	dreamer_weak_seed42_20260124_141543_final.pth	0.0490	0.0385	0.1960	0.0250	0.0125	0.0563	0.0630